

MOVIE STREAMING DATABASE

FUNCTIONAL DEPENDENCIES AND NORMALIZATION

This document lists the Functional Dependencies (FDs) identified for every relation in the Movie Streaming Database, derived from the ER diagram and table schema, and provides a step-by-step normalization proof ($1NF \rightarrow 2NF \rightarrow 3NF$) for each relation. The resulting table structure is shown explicitly after every normalization step, ending with the final normalized schema for each table.

Functional Dependency notation: $X \rightarrow Y$ means attribute(s) X functionally determine attribute(s) Y — for any two tuples agreeing on X, they must also agree on Y.

Normalization rules applied

- 1NF: All attributes hold atomic (indivisible) values; no repeating groups or multi-valued attributes; each row is uniquely identifiable.
- 2NF: The relation is in 1NF, and every non-key attribute is fully functionally dependent on the whole primary key (no partial dependency on part of a composite key).
- 3NF: The relation is in 2NF, and no non-key attribute is transitively dependent on the primary key (no non-key attribute determines another non-key attribute).

1. Table: USERS

- Attributes: user_id (PK), username, email, password, signup_date

Functional Dependencies

Determinant (LHS)	Dependent Attributes (RHS)
user_id	username, email, password, signup_date
username	user_id, email, password, signup_date
email	user_id, username, password, signup_date

Normalization Steps

Step 1 — 1NF Check — Satisfied

- Every attribute (username, email, password, signup_date) holds a single atomic value.
- No repeating groups or multi-valued columns exist.
- user_id uniquely identifies each row.

Resulting Table after 1NF — USERS

Attribute	Key	Depends On
user_id	PK	-
username	-	user_id
email	-	user_id
password	-	user_id
signup_date	-	user_id

Step 2 — 2NF Check — Satisfied

- Primary key (user_id) is a single atom (not composite), so partial dependency cannot occur by definition.
- All non-key attributes fully depend on the whole key, user_id.

Resulting Table after 2NF — USERS

Attribute	Key	Depends On
user_id	PK	-
username	-	user_id
email	-	user_id
password	-	user_id
signup_date	-	user_id

Step 3 — 3NF Check — Satisfied

- No non-key attribute determines another non-key attribute (e.g. email does not determine signup_date).
- $\therefore$ USERS is in 3NF (and BCNF, since every determinant — user_id, username, email — is a candidate key).

Resulting Table after 3NF — USERS

Attribute	Key	Depends On
user_id	PK	-
username	-	user_id
email	-	user_id
password	-	user_id
signup_date	-	user_id

Final Normalized Table — USERS (3NF / BCNF)

USERS — Final Schema

Attribute	Key	Depends On
user_id	PK	-
username	-	user_id
email	-	user_id
password	-	user_id
signup_date	-	user_id

- username and email are declared UNIQUE, so each independently determines the rest of the tuple, making them candidate keys alongside user_id.

2. Table: MOVIES

- Attributes: movie_id (PK), title, release_year, duration, language, rating

Functional Dependencies

Determinant (LHS)	Dependent Attributes (RHS)
movie_id	title, release_year, duration, language, rating

Normalization Steps

Step 1 — 1NF Check — Satisfied

- All attributes are atomic: title (text), release_year (number), duration (number), language (text), rating (text).
- movie_id uniquely identifies each row.

Resulting Table after 1NF — MOVIES

Attribute	Key	Depends On
movie_id	PK	-
title	-	movie_id
release_year	-	movie_id
duration	-	movie_id
language	-	movie_id
rating	-	movie_id

Step 2 — 2NF Check — Satisfied

- Primary key (movie_id) is a single attribute, so no partial dependency is possible.
- All non-key attributes fully depend on movie_id.

Resulting Table after 2NF — MOVIES

Attribute	Key	Depends On
movie_id	PK	-
title	-	movie_id
release_year	-	movie_id
duration	-	movie_id
language	-	movie_id
rating	-	movie_id

Step 3 — 3NF Check — Satisfied

- No transitive dependency exists: language does not determine rating, duration does not determine release_year.

- ∴ MOVIES is in 3NF (and BCNF).

Resulting Table after 3NF — MOVIES

Attribute	Key	Depends On
movie_id	PK	-
title	-	movie_id
release_year	-	movie_id
duration	-	movie_id
language	-	movie_id
rating	-	movie_id

Final Normalized Table — MOVIES (3NF / BCNF)

MOVIES — Final Schema

Attribute	Key	Depends On
movie_id	PK	-
title	-	movie_id
release_year	-	movie_id
duration	-	movie_id
language	-	movie_id
rating	-	movie_id

3. Table: GENRES

- Attributes: genre_id (PK), genre_name

Functional Dependencies

Determinant (LHS)	Dependent Attributes (RHS)
genre_id	genre_name

Normalization Steps

Step 1 — 1NF Check — Satisfied

- genre_name is a single atomic text value per row; no repeating groups.

Resulting Table after 1NF — GENRES

Attribute	Key	Depends On
genre_id	PK	-
genre_name	-	genre_id

Step 2 — 2NF Check — Satisfied

- Single-attribute primary key (genre_id) — partial dependency cannot arise.

Resulting Table after 2NF — GENRES

Attribute	Key	Depends On
genre_id	PK	-
genre_name	-	genre_id

Step 3 — 3NF Check — Satisfied

- Only one non-key attribute (genre_name); no transitive dependency possible.
- ∴ GENRES is in 3NF (and BCNF).

Resulting Table after 3NF — GENRES

Attribute	Key	Depends On
genre_id	PK	-
genre_name	-	genre_id

Final Normalized Table — GENRES (3NF / BCNF)

GENRES — Final Schema

Attribute	Key	Depends On
genre_id	PK	-
genre_name	-	genre_id

4. Table: MOVIE_GENRE (Junction / Bridge Table)

- Attributes: movie_id (PK/FK), genre_id (PK/FK)

Functional Dependencies

Determinant (LHS)	Dependent Attributes (RHS)
movie_id, genre_id	(no non-key attributes — pure junction table)

Normalization Steps

Step 1 — 1NF Check — Satisfied

- Both attributes (movie_id, genre_id) are atomic.
- The composite key (movie_id, genre_id) uniquely identifies each row (one row per movie-genre pairing).

Resulting Table after 1NF — MOVIE_GENRE (Junction / Bridge Table)

Attribute	Key	Depends On
movie_id	PK/FK	-
genre_id	PK/FK	-

Step 2 — 2NF Check — Satisfied (vacuously)

- There are no non-key attributes, so the partial-dependency rule has nothing to violate.

Resulting Table after 2NF — MOVIE_GENRE (Junction / Bridge Table)

Attribute	Key	Depends On
movie_id	PK/FK	-
genre_id	PK/FK	-

Step 3 — 3NF Check — Satisfied (vacuously)

- With no non-key attributes, no transitive dependency can exist.
- $\therefore$ MOVIE_GENRE is in 3NF (and BCNF).

Resulting Table after 3NF — MOVIE_GENRE (Junction / Bridge Table)

Attribute	Key	Depends On
movie_id	PK/FK	-
genre_id	PK/FK	-

Final Normalized Table — MOVIE_GENRE (Junction / Bridge Table) (3NF / BCNF)

MOVIE_GENRE (Junction / Bridge Table) — Final Schema

Attribute	Key	Depends On
movie_id	PK/FK	-

genre_id	PK/FK	-
-----------------	--------------	---

- This table exists purely to resolve the many-to-many relationship between MOVIES and GENRES and carries no descriptive attributes of its own.

5. Table: SUBSCRIPTION

- Attributes: subscription_id (PK), user_id (FK), plan, start_date, end_date

Functional Dependencies

Determinant (LHS)	Dependent Attributes (RHS)
subscription_id	user_id, plan, start_date, end_date

Normalization Steps

Step 1 — 1NF Check — Satisfied

- All attributes hold single atomic values (plan is one text value, dates are single date values).

Resulting Table after 1NF — SUBSCRIPTION

Attribute	Key	Depends On
subscription_id	PK	-
user_id	FK	subscription_id
plan	-	subscription_id
start_date	-	subscription_id
end_date	-	subscription_id

Step 2 — 2NF Check — Satisfied

- Primary key (subscription_id) is a single attribute, so no partial dependency is possible.
- All non-key attributes fully depend on subscription_id.

Resulting Table after 2NF — SUBSCRIPTION

Attribute	Key	Depends On
subscription_id	PK	-
user_id	FK	subscription_id
plan	-	subscription_id
start_date	-	subscription_id
end_date	-	subscription_id

Step 3 — 3NF Check — Satisfied

- user_id does NOT determine plan/start_date/end_date, since a user may have multiple subscription rows over time (renewals, plan changes) — so this is not a functional dependency and not a transitive-dependency risk.
- No non-key attribute determines another non-key attribute.
- ∴ SUBSCRIPTION is in 3NF (and BCNF).

Resulting Table after 3NF — SUBSCRIPTION

Attribute	Key	Depends On
subscription_id	PK	-
user_id	FK	subscription_id
plan	-	subscription_id
start_date	-	subscription_id
end_date	-	subscription_id

Final Normalized Table — SUBSCRIPTION (3NF / BCNF)

SUBSCRIPTION — Final Schema

Attribute	Key	Depends On
subscription_id	PK	-
user_id	FK	subscription_id
plan	-	subscription_id
start_date	-	subscription_id
end_date	-	subscription_id

6. Table: WATCH_HISTORY

- Attributes: history_id (PK), user_id (FK), movie_id (FK), watch_date, watched_minutes, completed

Functional Dependencies

Determinant (LHS)	Dependent Attributes (RHS)
history_id	user_id, movie_id, watch_date, watched_minutes, completed

Normalization Steps

Step 1 — 1NF Check — Satisfied

- All attributes are atomic (watch_date is a single datetime, watched_minutes a single number, completed a single boolean).

Resulting Table after 1NF — WATCH_HISTORY

Attribute	Key	Depends On
history_id	PK	-
user_id	FK	history_id
movie_id	FK	history_id
watch_date	-	history_id
watched_minutes	-	history_id
completed	-	history_id

Step 2 — 2NF Check — Satisfied

- Primary key (history_id) is a single attribute — partial dependency is not possible.
- All non-key attributes fully depend on history_id.

Resulting Table after 2NF — WATCH_HISTORY

Attribute	Key	Depends On
history_id	PK	-
user_id	FK	history_id
movie_id	FK	history_id
watch_date	-	history_id
watched_minutes	-	history_id
completed	-	history_id

Step 3 — 3NF Check — Satisfied

- No non-key attribute determines another non-key attribute within this relation.
- $\therefore$ WATCH_HISTORY is in 3NF (and BCNF).

Resulting Table after 3NF — WATCH_HISTORY

Attribute	Key	Depends On
history_id	PK	-
user_id	FK	history_id
movie_id	FK	history_id
watch_date	-	history_id
watched_minutes	-	history_id
completed	-	history_id

Final Normalized Table — WATCH_HISTORY (3NF / BCNF)

WATCH_HISTORY — Final Schema

Attribute	Key	Depends On
history_id	PK	-
user_id	FK	history_id
movie_id	FK	history_id
watch_date	-	history_id
watched_minutes	-	history_id
completed	-	history_id

- history_id is used as a surrogate key because a user may watch the same movie multiple times, so (user_id, movie_id) alone cannot serve as a candidate key.

7. Table: RATINGS

- Attributes: rating_id (PK), user_id (FK), movie_id (FK), score, review

Functional Dependencies

Determinant (LHS)	Dependent Attributes (RHS)
rating_id	user_id, movie_id, score, review
user_id, movie_id	score, review, rating_id

Normalization Steps

Step 1 — 1NF Check — Satisfied

- score is a single atomic numeric value; review is a single atomic text value (no repeating groups of reviews per row).

Resulting Table after 1NF — RATINGS

Attribute	Key	Depends On
rating_id	PK	-
user_id	FK	rating_id / (user_id, movie_id)
movie_id	FK	rating_id / (user_id, movie_id)
score	-	rating_id / (user_id, movie_id)
review	-	rating_id / (user_id, movie_id)

Step 2 — 2NF Check — Satisfied

- Taking rating_id as the primary key (single attribute), no partial dependency can occur.
- Under the alternate candidate key (user_id, movie_id), both score and review depend on the full pair, not on user_id or movie_id alone.

Resulting Table after 2NF — RATINGS

Attribute	Key	Depends On
rating_id	PK	-
user_id	FK	rating_id / (user_id, movie_id)
movie_id	FK	rating_id / (user_id, movie_id)
score	-	rating_id / (user_id, movie_id)

review	-	rating_id / (user_id, movie_id)
--------	---	---------------------------------

Step 3 — 3NF Check — Satisfied

- No non-key attribute determines another non-key attribute (score does not determine review or vice versa).
- $\therefore$ RATINGS is in 3NF (and BCNF).

Resulting Table after 3NF — RATINGS

Attribute	Key	Depends On
rating_id	PK	-
user_id	FK	rating_id / (user_id, movie_id)
movie_id	FK	rating_id / (user_id, movie_id)
score	-	rating_id / (user_id, movie_id)
review	-	rating_id / (user_id, movie_id)

Final Normalized Table — RATINGS (3NF / BCNF)

RATINGS — Final Schema

Attribute	Key	Depends On
rating_id	PK	-
user_id	FK	rating_id / (user_id, movie_id)
movie_id	FK	rating_id / (user_id, movie_id)
score	-	rating_id / (user_id, movie_id)
review	-	rating_id / (user_id, movie_id)

- Because the schema enforces UNIQUE(user_id, movie_id) — one rating per user per movie — the composite (user_id, movie_id) is also a valid candidate key alongside the surrogate rating_id.

CONCLUSION

Every relation in the Movie Streaming Database — USERS, MOVIES, GENRES, MOVIE_GENRE, SUBSCRIPTION, WATCH_HISTORY, and RATINGS — satisfies 1NF, 2NF, and 3NF:

- **1NF**: every attribute in every table stores a single atomic value; no repeating groups or multi-valued columns exist anywhere in the schema.
- **2NF**: every table's primary key is either a single attribute (making partial dependency structurally impossible) or, for MOVIE_GENRE, a composite key with no non-key attributes at all (satisfied vacuously).
- **3NF**: in every table, non-key attributes depend only on the primary key and never on each other — no transitive dependencies were found.

Since every functional dependency identified has a determinant that is a candidate key of its relation, the schema in fact satisfies the stronger BCNF (Boyce-Codd Normal Form) as well. The schema is therefore well-normalized, free of update, insertion, and deletion anomalies caused by redundant functional dependencies.